FACULDADE Donaduzzi

ENGENHARIA DE SOFTWARE

ORIENTAÇÃO A OBJETOS E UML

ERICKA RAYANNY

MARCOS GUERRA

NICOLAS MUCKE

PEDRO AUGUSTO

RAFAEL HACHBARTH

ROGÉRIO EMANUEL

Relatório sobre o uso de listas em Java

Sistema de clínica veterinária

Relatório sobre o uso de listas em Java

1. O que é ArrayList

ArrayList é uma classe do Java (pacote *java.util*) que guarda uma sequência de objetos em ordem, como uma lista. Ela cresce e diminui sozinha conforme adiciono ou removo elementos, então não preciso definir o tamanho antes. No projeto, ela fica dentro da classe *ClinicaVeterinaria:*

```
private ArrayList<Animal> animais = new ArrayList<>();
```

O <*Animal*> serve para indicar que a lista só aceita objetos do tipo Animal (e de suas subclasses).

2. Diferença entre vetor e *ArrayList*

O vetor tem tamanho fixo, enquanto o *ArrayList* aumenta ou diminui automaticamente. Além disso, o *ArrayList* oferece métodos que facilitam adicionar, remover e acessar elementos, tornando seu uso mais prático.

3. Como adicionar e remover elementos

Para adicionar, uso add, que coloca o objeto no final da lista:

```
animais.add(animal);
```

Para remover, uso remove, que pode receber a posição ou o próprio objeto:

```
animais.remove(animal);
animais.remove(0);
```

No projeto, o método *adicionar()* verifica antes se o código já existe, e o método *remover()* busca o animal pelo código e depois o remove da lista.

4. Como percorrer a lista com *for* e *for-each*

Com o for comum, uso um contador e acesso cada posição com *get:*

```
for (int i = 0; i < animais.size(); i++) {
    System.out.println(animais.get(i).getDescricao());
}
```

Com o *for-each*, o Java entrega cada elemento direto, sem precisar de contador:

```
for (Animal animal : animais) {
    System.out.println(animal.getNome() + " faz: " +
animal.emitirSom());
}
```

No projeto, usamos o for em *listar() e o for-each em buscar() e exibirSons().*

5. Como verificar o tamanho da lista

O método *size()* devolve a quantidade de elementos. O método *isEmpty()* diz se a lista está vazia. Usei os dois em *listar():*

```
if (animais.isEmpty()) {
    System.out.println("Nenhum animal cadastrado.");
}
System.out.println("Total de animais: " + animais.size());
```

6. Como buscar um objeto por código ou identificador

O *ArrayList* não busca por código sozinho. Preciso percorrer a lista e comparar o código de cada objeto com o que procuro. Se achar, devolvo o objeto; se terminar o laço sem achar, lanço uma exceção:

```
public Animal buscar(int codigo) {
    for (Animal animal : animais) {
        if (animal.getCodigo() == codigo) {
            return animal;
        }
    }
    throw new IllegalArgumentException("Nenhum animal encontrado
com o código " + codigo + ".");
}
```

7. O que ocorre ao tentar acessar uma posição inexistente

Se tentar acessar uma posição que não existe na lista, o Java apresenta um erro. Por isso, é importante verificar se a posição é válida antes de acessar. No projeto, usamos o código do animal para fazer a busca, evitando esse tipo de problema.

8. Por que uma lista da superclasse pode armazenar objetos de subclasses diferentes

Como *Cao* e *Gato* herdam de *Animal,* podemos colocar os dois na mesma lista. Cada um mantém suas próprias características, como o som que faz: o cão late e o gato mia. Assim, fica mais fácil organizar os animais e adicionar novas espécies no futuro.